Atividade: Reconhecimento de Gestos com Deep Learning

Machine Learning e Deep Learning - Profa. Thais Amanda Santos
Eduardo Sochodolak e Andrew Bertelli - 27/10/2026

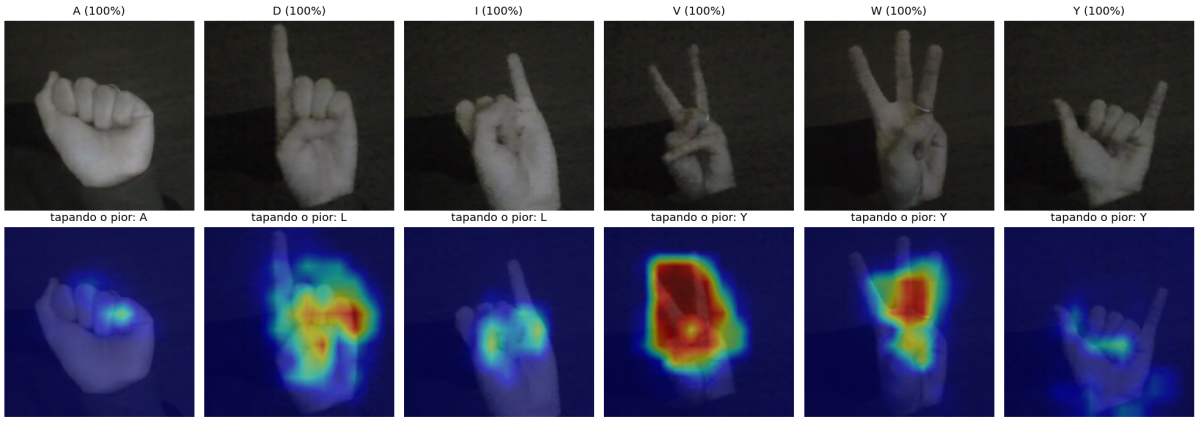
Escolhemos 10 letras do alfabeto manual de Libras que não têm movimento: A, B, C, D, I, L, O, V, W e Y, mais a classe nada. Cada classe tem 240 fotos, 120 com fundo claro e 120 com fundo escuro. O modelo final acertou 93,6% das fotos de validação.

1. Esse modelo é um deep learning ou um machine learning?

É deep learning. O modelo usa a MobileNetV2, uma rede neural convolucional com 52 camadas de convolução e cerca de 2,2 milhões de parâmetros, já treinada na ImageNet. A gente não escolheu quais características usar: a rede aprende sozinha a partir dos pixels, começando por bordas nas primeiras camadas e chegando a formas mais complexas nas últimas. Num machine learning clássico teríamos que extrair as características na mão, como contar os dedos esticados, e passar esses números para um SVM ou uma árvore de decisão. Mesmo com a base congelada e só a última camada sendo treinada (14.091 parâmetros), toda previsão passa pela rede inteira, então continua sendo deep learning.

2. Que características da imagem o modelo usa para distinguir os gestos?

Nossa hipótese é que o modelo olha para os dedos: quais estão esticados, quantos são e onde estão. A matriz de confusão já apontava isso, porque os erros aconteciam entre sinais que mudam por um dedo, como D e I ou V e W. Para testar, passamos um quadrado cinza por cada parte da foto e medimos quanto a confiança na classe certa caía (experimentos/oclusao.py). As regiões que mais derrubaram a confiança ficaram sempre em cima da mão. No D, tapar os dedos dobrados derrubou a confiança em 95% e o modelo passou a dizer L. No V e no W, tapar a base dos dedos levantados derrubou quase 100%. Tapar o fundo praticamente não mudou nada, o que confirma a hipótese. Também espelhamos as fotos para simular a outra mão e a acurácia ficou igual (93,6% para 93,8%), então o modelo não depende de qual mão faz o sinal.



3. Que decisões na coleta de dados ajudam ou prejudicam o modelo?

A primeira foi coletar em duas iluminações. O modelo da primeira coleta, treinado só com fundo claro, acertou apenas 39,5% das fotos de fundo escuro, confundindo por exemplo Y com L e D com B. Treinando com as duas iluminações, o modelo final acertou 100% das

fotos escuras e 87,1% das claras da validação.

A segunda foi a ordem da coleta. Na primeira versão gravamos cada sinal em 30 segundos seguidos, e no fim de alguns sinais entrou a mão de outra pessoa, mais perto da câmera. Como o train.py usa os últimos 20% de cada classe como validação, essa mão ficou só na validação e o D foi confundido com I em 35 das 60 fotos. Na pedra, papel e tesoura também vimos que deixar gravando com a mão fora do quadrado gera fotos de pedra sem mão nenhuma, e nelas o modelo respondia nada.

4. Como o modelo se comporta se você treinar com dados que têm pessoas no fundo?

Coletamos os 11 sinais de novo numa sala de aula, com pessoas sentadas e passando ao fundo, e treinamos com o mesmo train.py. O primeiro treino deu 79,9%, mas era o mesmo erro da pedra: a gravação continuou depois que a mão saiu e 41 fotos de D, V e Y estavam sem mão. Tirando essas fotos, o modelo acertou 94% da validação, só que treino e validação eram do mesmo lugar. Testado nas nossas fotos sem pessoas, ele acertou só 20,8% e respondia B para quase tudo, até sem mão nenhuma. Já o modelo treinado sem pessoas, testado nas fotos com pessoas, acertou 33,5% e nunca respondeu nada quando só tinha gente no fundo: chutou C, O ou W nas 26 fotos.

O modelo olha para o quadrado inteiro, não só para a mão, então ele aprende o ambiente junto: a sala, a parede e as pessoas. Rosto e braços de quem está atrás parecem mão, então ele vê alguma coisa e chuta um sinal. Como os testes cruzados mudam o lugar e as pessoas ao mesmo tempo, parte da queda vem do lugar novo, mas o caso do nada mostra o efeito das pessoas. Duas formas de melhorar: recortar só a mão antes de classificar, usando um detector de mão como o MediaPipe, e coletar todas as classes, inclusive nada, com e sem pessoas atrás, para o fundo não ficar ligado a nenhum sinal.